Lesson 26

26.03.2026

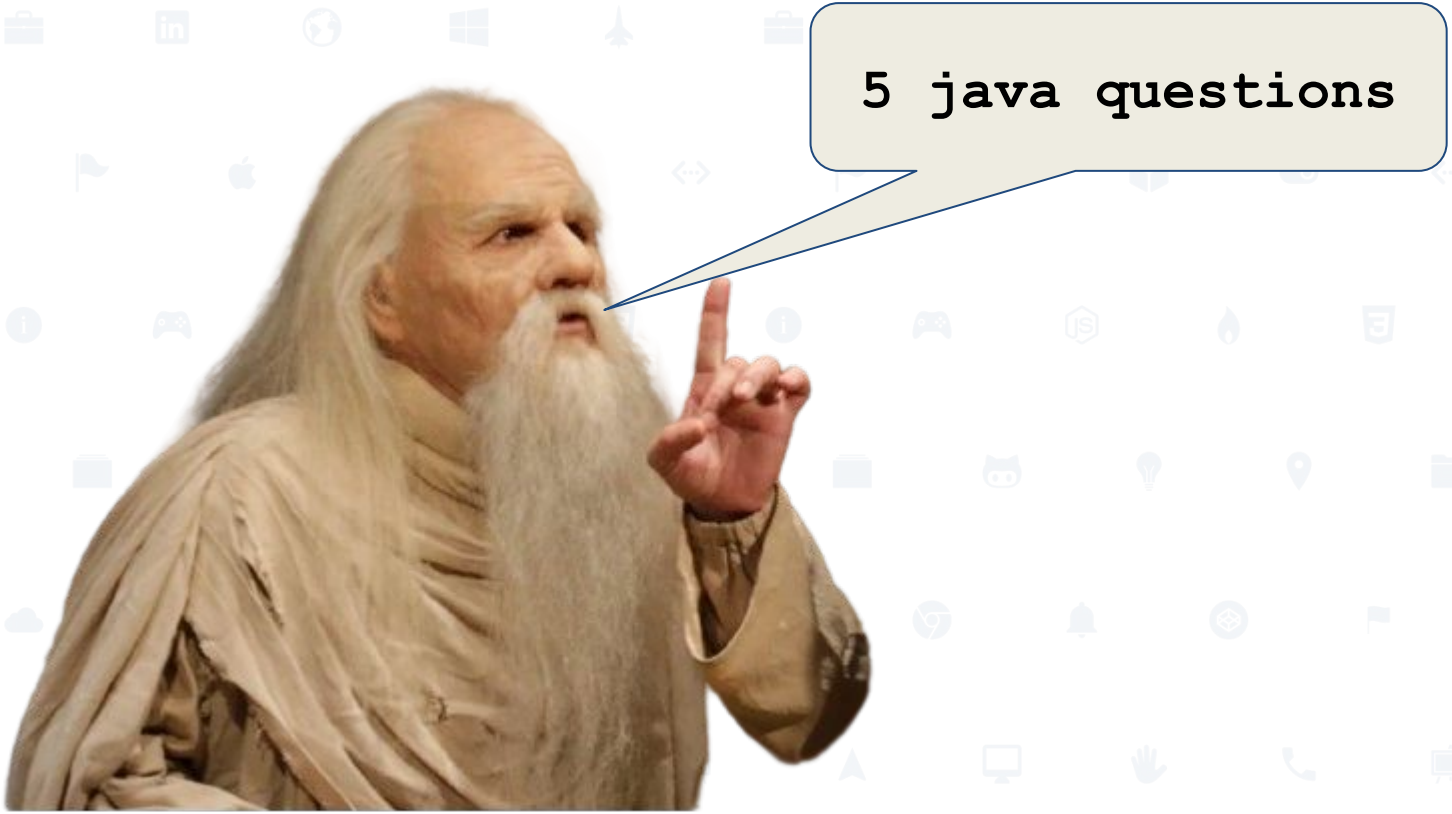
```
public class ex1 {
    int num = 100;
    public void calc(int num)   { num = num * 10;   }
    public void printNum()      { System.out.println(num); }

    public static void main(String[] args)
    {
        ex1 obj = new ex1();
        obj.calc(2);
        obj.printNum();
    }
}
```

```java
class First {
    public First() { System.out.println("a"); }
}

class Second extends First {
    public Second() { System.out.println("b"); }
}

class Third extends Second {
    public Third() { System.out.println("c"); }
}

public class ex2 {
    public static void main(String[] args) {
        Third c = new Third();
    }
}
```

```
public class ex3 {
    public static void main(String[] args) {
        var list = List.of(new String[]{"A", "BB", "CCC"},
                new String[]{"DD", "E"});
        list.forEach(x ->
                System.out.print(x.length));
    }
}
```

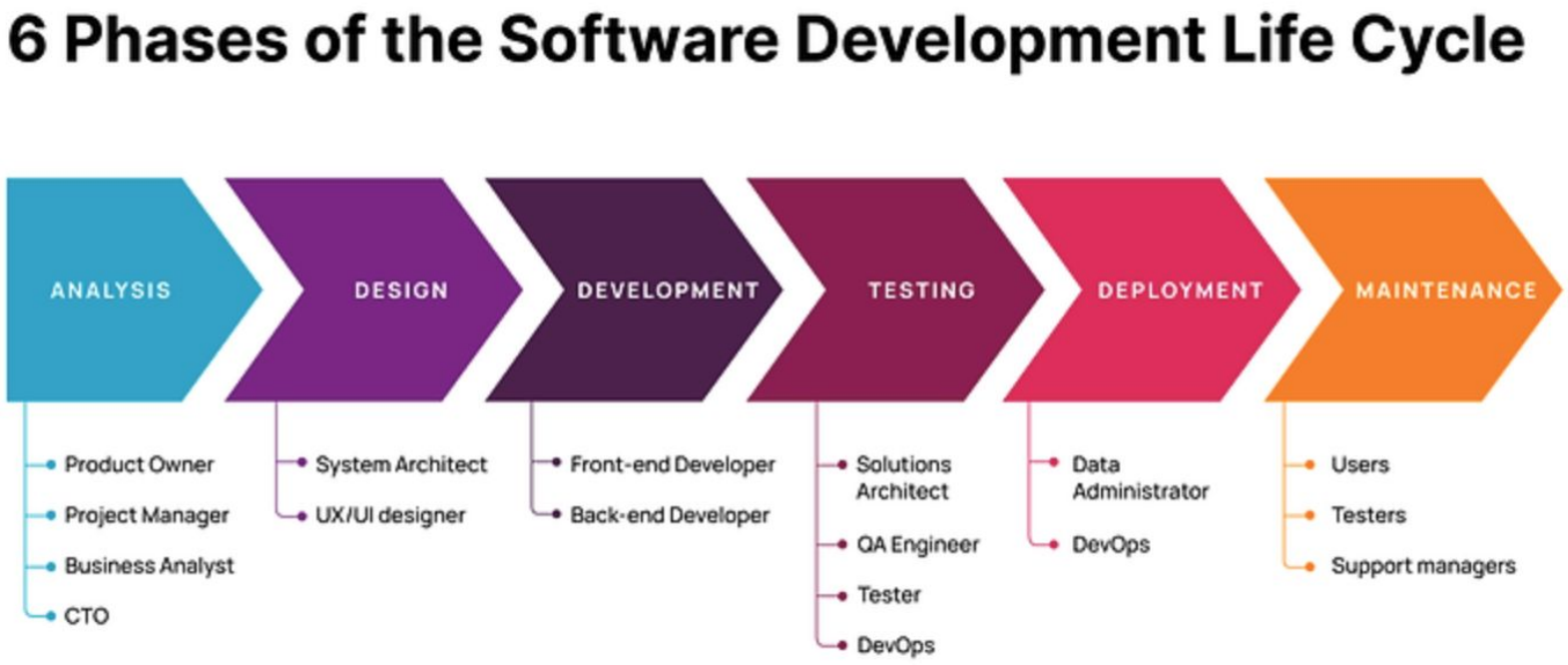
```java
public class ex4 {
    public static void main(String[] args) {
        char[][] arr = {
                {'A', 'B', 'C'},
                {'D', 'E', 'F'},
                {'G', 'H', 'I'}
        };
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++) {
                System.out.print(arr[i][1]);
            }
            System.out.println();
        }
    }
}
```

```
class Car {
    void speed(Byte val) {System.out.println("DARK");}

    void speed(byte... vals) {System.out.println("LIGHT");}
}

public class ex5 {

    public static void main(String[] args) {
        byte b = 10;
        new Car().speed(b);
    }
}
```


5 java questions

1. Відмінності String/StringBuilder/StringBuffer
2. Interface vs Abstract Class
3. override vs overload
4. Регіони пам'яті в JVM
5. java.util.collection.

Методологія розробки програмного забезпечення (англ. Software development methodology) — сукупність методів, застосовуваних на різних стадіях забезпечення, що мають спільний філософський підхід та, відповідно до цього підходу, дозволяють забезпечити найкращу ефективність процесів розробки.



Agile Modeling

Набір концепцій, принципів і методів (практик), який дозволяє швидко і легко виконувати проектування та документацію для проектів з розробки програмного забезпечення. Не включає в себе докладні інструкції з проектування, містить описи, як побудувати діаграми UML. Основна мета – ефективне моделювання та документація, але не включає в себе програмування і тестування, управління проектом, розгортання та обслуговування системи.

DSDM

Грунтується на концепції швидкої розробки додатків (Rapid Application Development, RAD). Це ітеративний і інкрементний підхід, який підкреслює постійну участь користувача/споживача в проекті.

Extreme programming (XP)

Ідея полягає в тому, щоб використовувати корисні традиційні методи розробки по-новому. Наприклад, код що написаний одним девелопером, перевіряється іншим. Також дуже часто використовують парне програмування, де один кодер пише код, а його партнер цей код одразу перевіряє.

Feature driven development (FDD)

Функціонально-орієнтована розробка. Поняття FDD дуже близькі до RUP, єдина різниця: “Кожна функція повинна реалізовуватися не більше двох тижнів.” Якщо завдання досить невелике, його можна розглядати як окрему фунцію. Якщо завдання є великим, то він повинен бути розділений на декілька відносно самостійні функції.

Scrum

Встановлює правила для процесу управління розробки програмного забезпечення і дозволяє використовувати існуючу практику кодування, регулювати вимоги або приймати тактичні зміни. Використовуючи цю методику можливо виявити і усунути відхилення від бажаного результату на більш ранніх стадіях розробки програмного забезпечення.

Lean software development

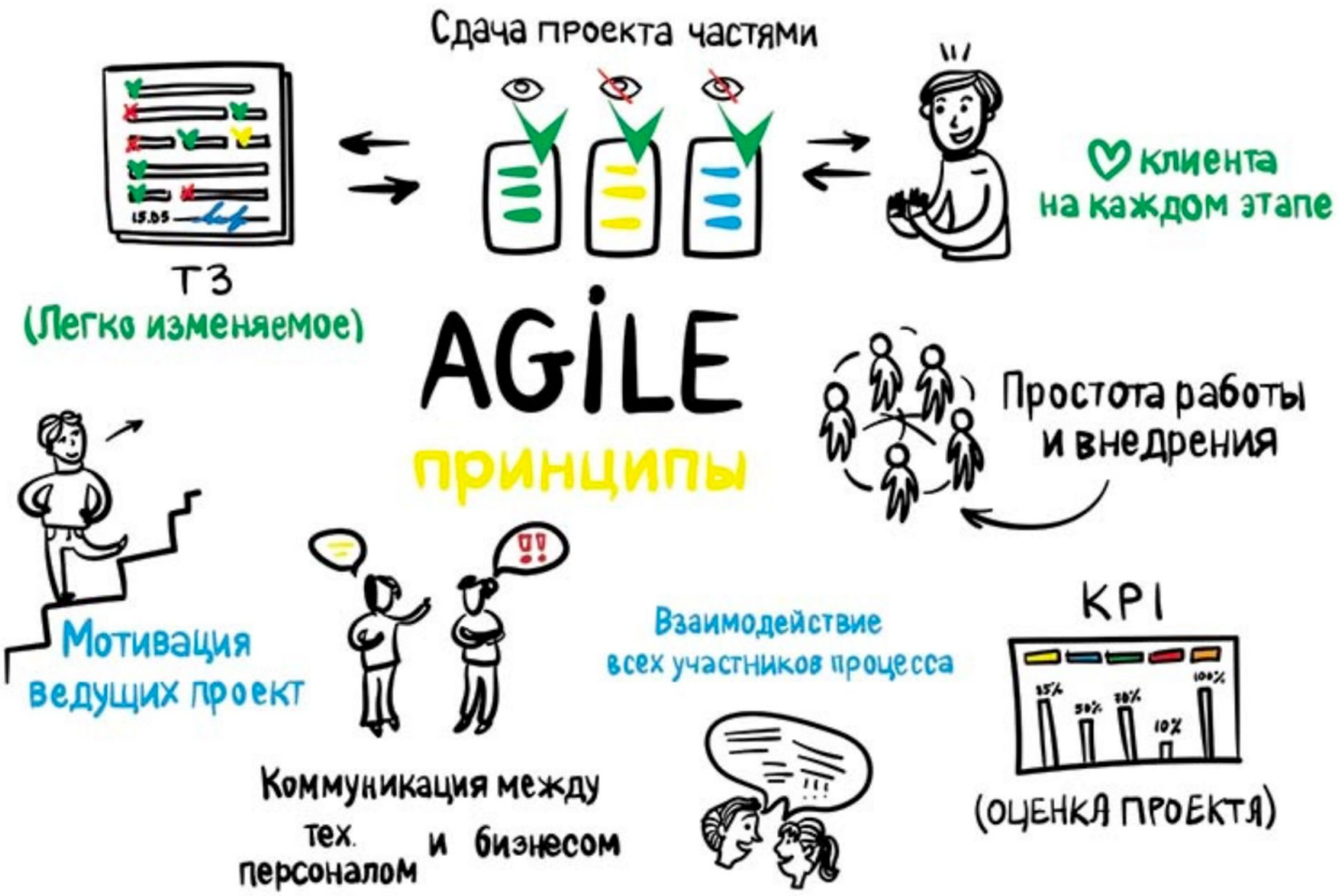
Ця методологія розробки програмного забезпечення метод бережливого виробництва (Lean Manufacturing). Основні принципи цієї методології є: максимальна швидка доставка продукції замовнику з дуже короткими ітераціями, мотивація команди і зосередитися на навчанні з короткими циклами розвитку, раннє тестування і зворотній зв’язок з клієнтами.

Kanban software development

Канбан реалізує принцип «точно в строк» і урівноважує робоче навантаження між усіма членами команди. За допомогою цього методу весь процес розробки зрозумілий для всіх членів команди. Канбан є візуальною моделлю розвитку, яка показує те, що потрібно виробляти, коли і скільки.

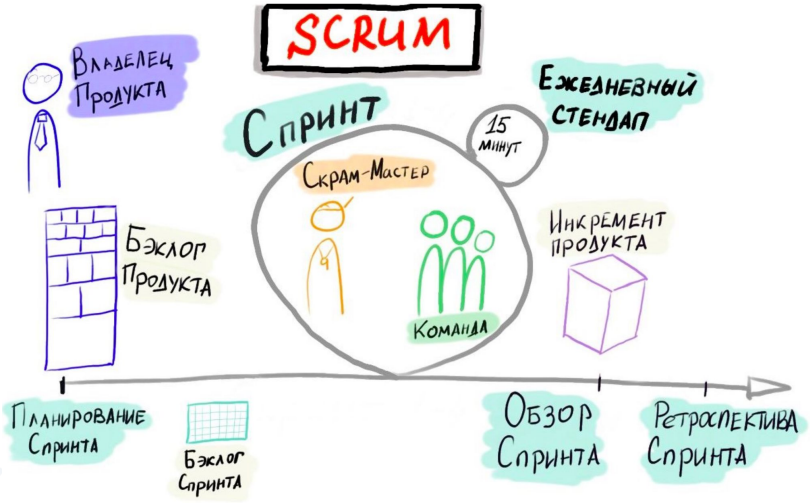
Scrumban

Scrumban є структурою управління і гібрид Scrum і Kanban. Розробники працюють з історіями користувачів і намагаються зберігати ітерації якомога коротшими. Тут не існує конкретних ролей, як наприклад в Scum. Кожен член команди зберігає свою існуючу роль в проекті.


Сдача проекта частями
ТЗ
(Легко изменяемое)
клиента
на каждом этапе
AGILE
принципы
Простота работы
и внедрения
Мотивация
ведущих проект
Взаимодействие
всех участников процесса
KPI
(ОЦЕНКА ПРОЕКТЯ)
Коммуникация между
тех. персоналом и бизнесом

Методологія "Scrum"

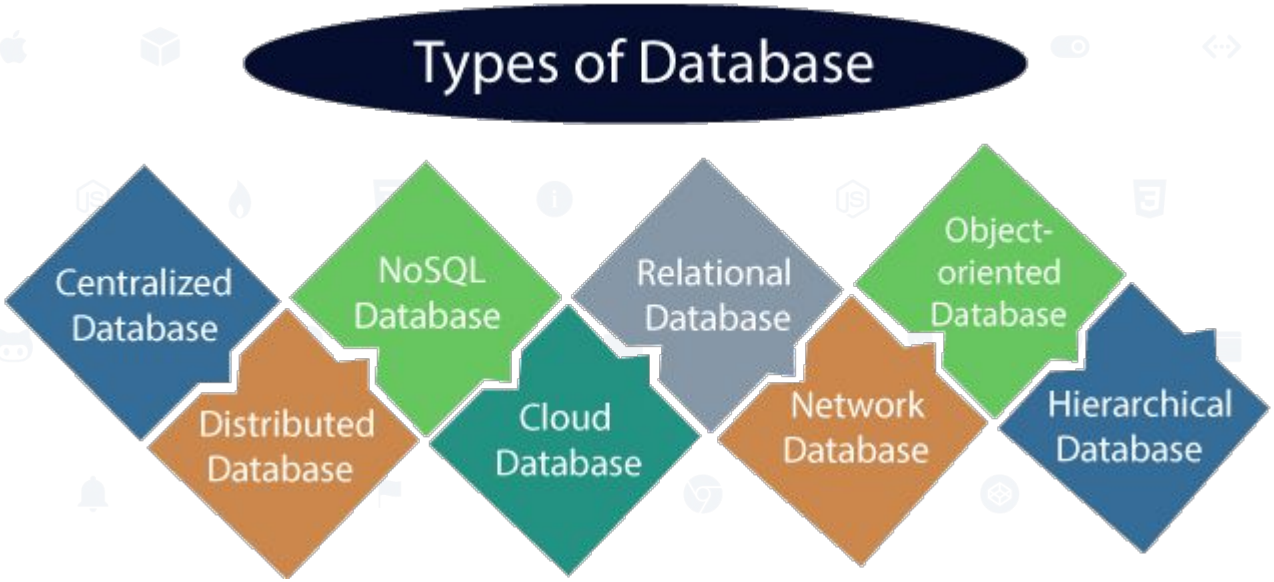
Scrum - гнучкий метод управління проектами. В його основі лежать «спринти» -короткі ітерації, суворо обмежені в часі (зазвичай, 2-4 тижня). Тривалість нарад при цьому зводиться до мінімуму, але збільшується їхня частота. Кожен спринт складається зі списку завдань до кінця ітерації, і кожна операція має свою “вагу”. Під час нарад команда обговорює, хто що зробив, що збирається зробити і які проблеми. Для планування у Scrum використовують журнал спринтів. У такому підході в команді часто з'являється Scrum майстер, який налагоджує безперервну роботу всієї команди, створюючи для її комфортні умови. Також на проекті з'являється роль Product Owner. керівника розробки, людини, яка стежить за продуктом та виступає головною ланкою між запитом клієнта та результатом команди

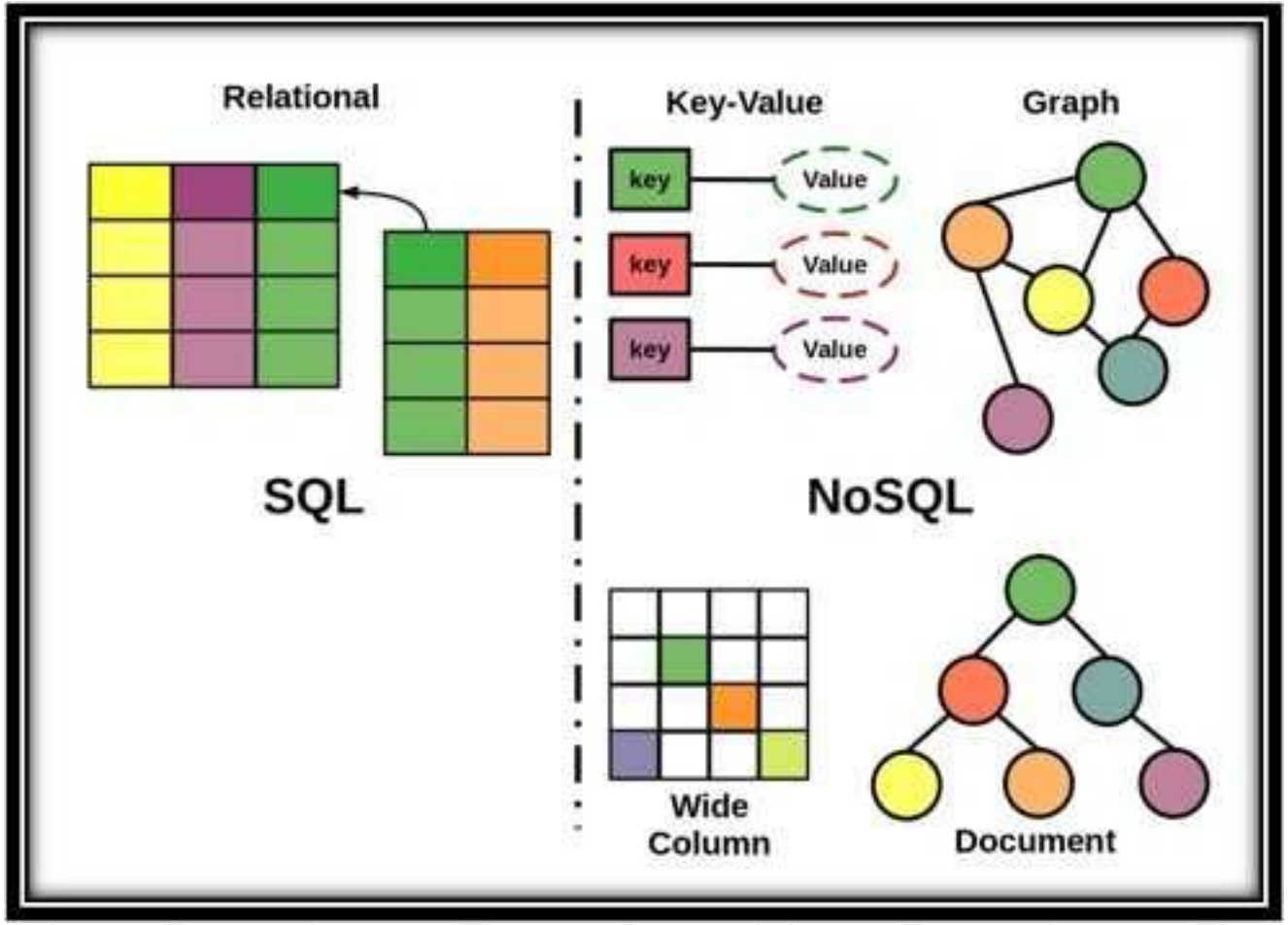


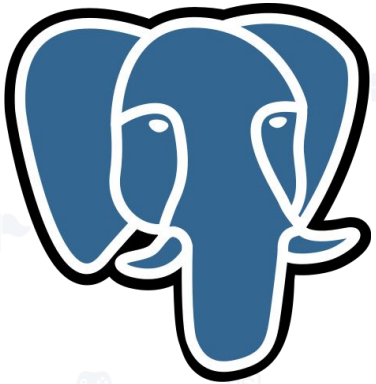
Методологія «Kanban»

Найважливішою особливістю Kanban є візуалізація життєвого циклу проекту. Створюються стовпчики виконання завдань, які здаються індивідуально. Колонки позначені маркерами типу: To do, In progress, Code review, In testing, Done (назва колонок, звичайно ж, може змінюватися). Мета кожного учасника команди - зменшувати кількість завдань у першій колонці. Підхід Kanban наочний і допомагає зрозуміти де виникла проблема. Структуру Kanban не визначають остаточно і безповоротно: залежно від специфіки проекту можна додати імпровізовані колонки. Наприклад, деякі команди використовують систему, у якій потрібно визначити критерії готовності завдання перед виконанням. Тоді додають дві колонки – specify (уточнити параметри) та execute (взятися за роботу)

База даних - сукупність даних, що зберігаються відповідно до схеми даних, маніпулювання якими виконують відповідно до правил засобів моделювання даних.




Relational
SQL
Key-Value
key
Value
key
Value
key
Value
Graph
NoSQL
Wide
Column
Document



MariaDB

Microsoft®
SQL Server®

SQLite

H2

MySQL™

ORACLE®
DATABASE

https://db-engines.com/en/ranking

429 systems in ranking, March 2026

Rank			DBMS	Database Model	Score		
Mar 2026	Feb 2026	Mar 2025			Mar 2026	Feb 2026	Mar 2025
1.	1.	1.	Oracle	Relational, Multi-model	1182.46	-21.05	-70.62
2.	2.	2.	MySQL	Relational, Multi-model	858.34	-9.88	-129.79
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	711.47	+3.33	-76.67
4.	4.	4.	PostgreSQL	Relational, Multi-model	680.08	+8.05	+16.66
5.	5.	5.	MongoDB	Document, Multi-model	383.58	+4.85	-12.85
6.	6.	6.	Snowflake	Relational	211.24	+3.10	+49.46
7.	↑ 8.	↑ 13.	Databricks	Multi-model	145.81	+1.29	+49.80
8.	↓ 7.	↓ 7.	Redis	Key-value, Multi-model	145.19	-1.85	-10.17
9.	9.	9.	IBM Db2	Relational, Multi-model	111.38	+0.16	-15.19
10.	10.	↓ 8.	Elasticsearch	Multi-model	103.58	-2.88	-27.80
11.	11.	11.	Apache Cassandra	Wide column, Multi-model	101.88	+0.28	-4.78
12.	12.	↓ 10.	SQLite	Relational	95.97	-3.22	-17.11
13.	13.	↑ 14.	MariaDB	Relational, Multi-model	87.00	+0.91	-7.23
14.	14.	↑ 17.	Microsoft Azure SQL Database	Relational, Multi-model	73.92	+0.26	+0.01
15.	↑ 16.	↑ 18.	Apache Hive	Relational	72.95	-0.05	+2.87
16.	↓ 15.	↓ 15.	Splunk	Search engine	72.21	-0.84	-6.66
17.	17.	↓ 12.	Microsoft Access	Relational	67.56	-1.90	-29.15
18.	18.	↓ 16.	Amazon DynamoDB	Multi-model	65.42	-2.46	-11.16
19.	19.	19.	Google BigQuery	Relational	55.82	-3.76	-2.14
20.	20.	20.	Neo4j	Graph	47.32	-1.94	+1.17

Sql database structure

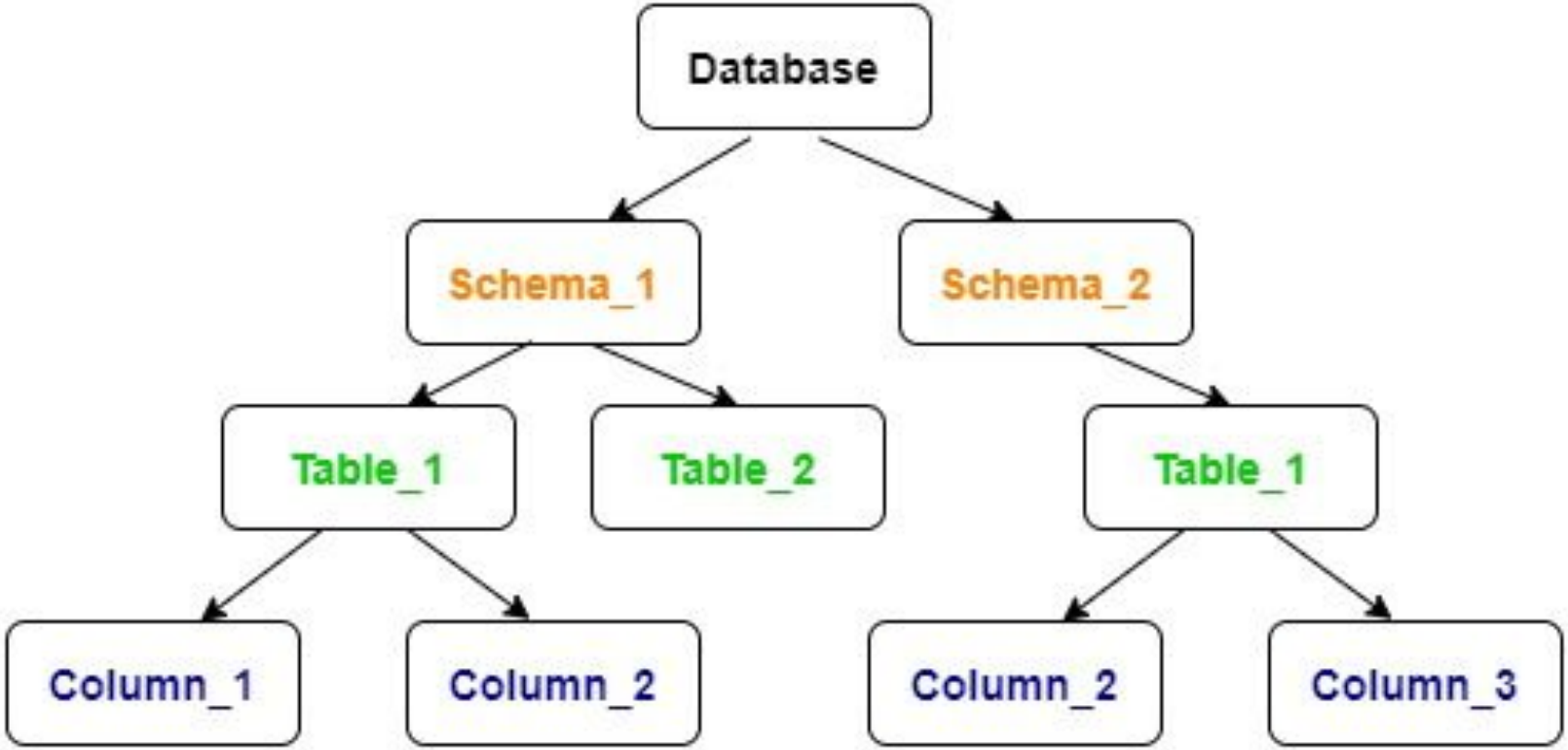
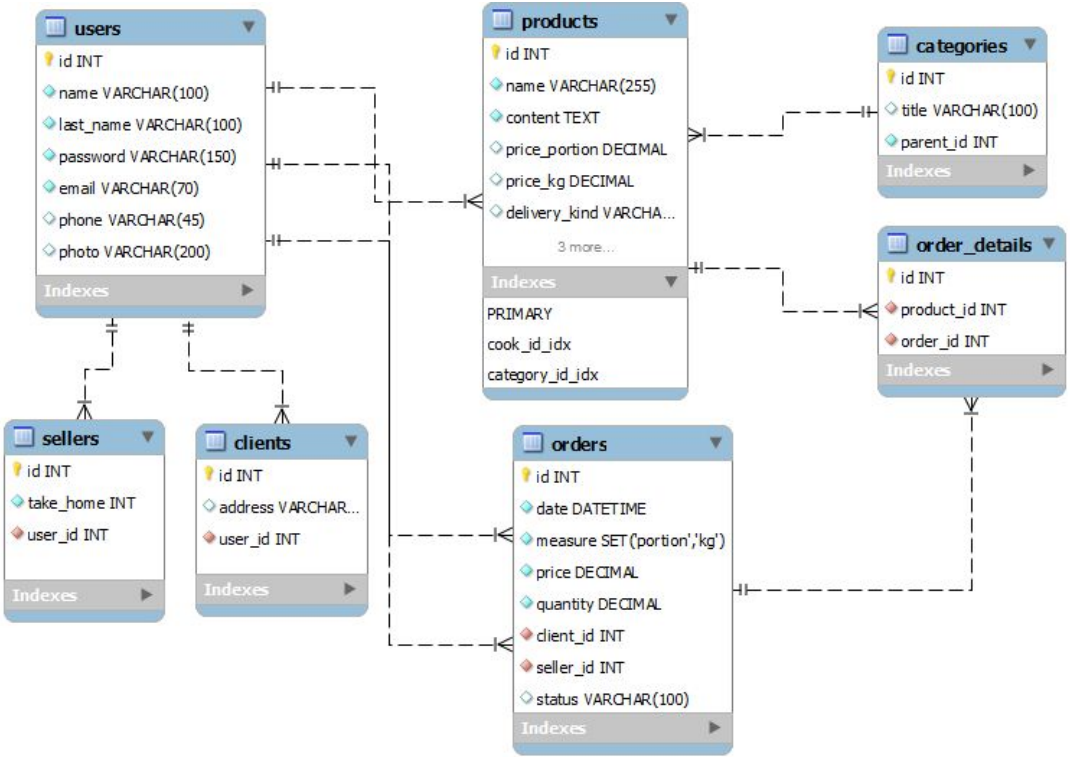
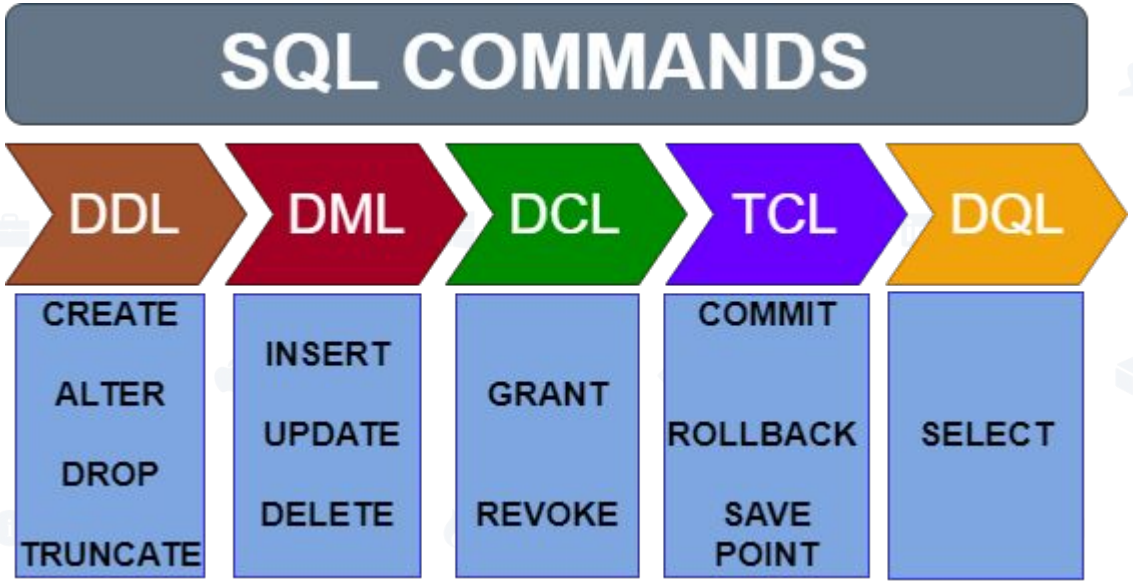


Схема бази даних





1. DDL – Data Definition Language
2. DQL – Data Query Language
3. DML – Data Manipulation Language
4. DCL – Data Control Language
5. TCL – Transaction Control Language

- **CREATE**: This command is used to create the database or its objects (like table, index, function, views, store procedure, and triggers).
- **DROP**: This command is used to delete objects from the database.
- **ALTER:** This is used to alter the structure of the database.
- **TRUNCATE:** This is used to remove all records from a table, including all spaces allocated for the records are removed.
- **COMMENT**: This is used to add comments to the data dictionary.
- **RENAME:** This is used to rename an object existing in the database.

- **SELECT:** It is used to retrieve data from the database.

- **INSERT**: It is used to insert data into a table.
- **UPDATE:** It is used to update existing data within a table.
- **DELETE**: It is used to delete records from a database table.
- **LOCK:** Table control concurrency.
- **CALL:** Call a PL/SQL or JAVA subprogram.
- **EXPLAIN PLAN:** It describes the access path to data.

```
CREATE TABLE [schema.]table
        (column datatype [DEFAULT expr][, ...]);
```

Data Type	Description
VARCHAR2(*size*)	Variable-length character data
CHAR(*size*)	Fixed-length character data
NUMBER(*p*,*s*)	Variable-length numeric data
DATE	Date and time values
LONG	Variable-length character data (up to 2 GB)
CLOB	Character data (up to 4 GB)
RAW and LONG RAW	Raw binary data
BLOB	Binary data (up to 4 GB)
BFILE	Binary data stored in an external file (up to 4 GB)
ROWID	A base-64 number system representing the unique address of a row in its table

```sql
ALTER TABLE table_name
ADD column_name datatype;
```

```sql
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

```sql
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

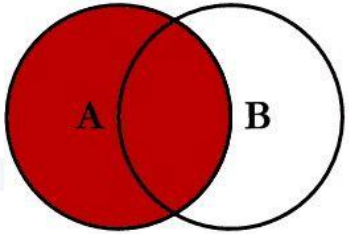
```sql
SELECT column1, column2, ...
FROM table_name;
```

```sql
SELECT COUNT(column_name)
FROM table_name
WHERE condition;
```

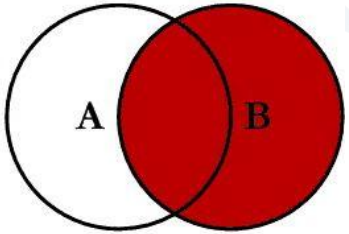
```sql
SELECT AVG(column_name)
FROM table_name
WHERE condition;
```

```sql
SELECT SUM(column_name)
FROM table_name
WHERE condition;
```

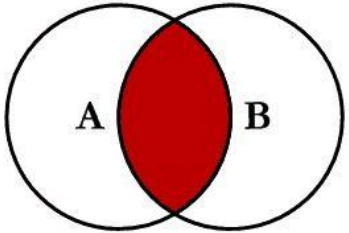
SQL JOINS



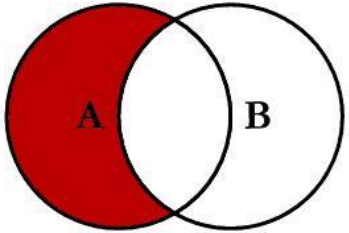

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



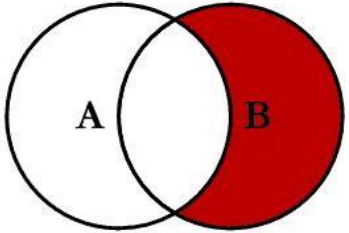

```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



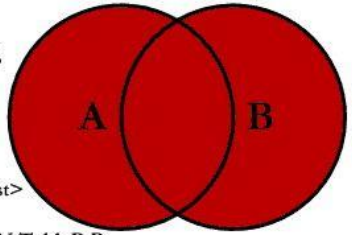

```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



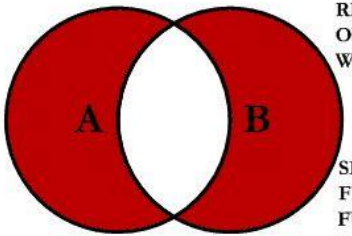

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```

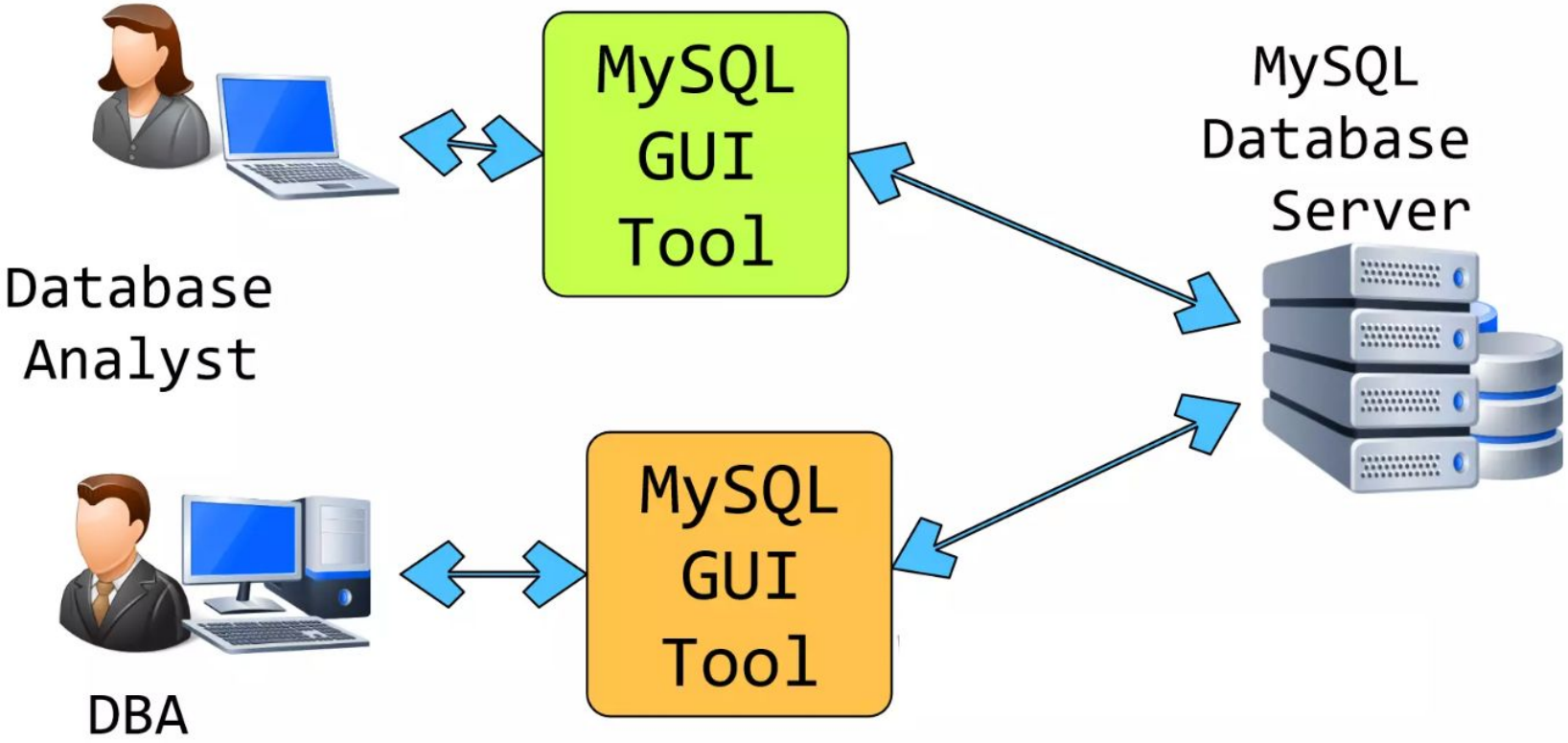



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

© C.L. Moffatt, 2008


MySQL
GUI
Tool
MySQL
Database
Server
Database
Analyst
MySQL
GUI
Tool
DBA